

SparkSession vs SparkContext

What is SparkSession?

SparkSession is the **entry point** to all Spark functionality in Spark 2.x and later. It provides a single unified interface for working with:

- DataFrames
- Spark SQL
- Structured Streaming
- Machine Learning (MLlib)
- SparkContext (RDD operations)

Instead of creating multiple contexts (SQLContext, HiveContext, SparkContext), SparkSession combines them into one object.

Example

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MyApp") \
    .getOrCreate()
```

What is SparkContext?

SparkContext is the **core engine** of Apache Spark. It establishes the connection between your application and the Spark cluster and is responsible for executing Spark jobs.

SparkContext is mainly used for **RDD (Resilient Distributed Dataset)** operations.

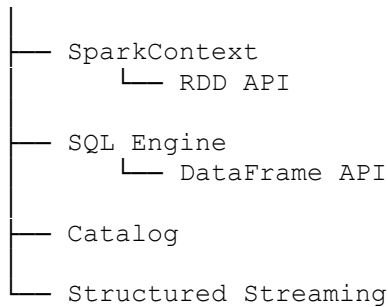
Example

```
sc = spark.sparkContext

rdd = sc.parallelize([1, 2, 3, 4])
print(rdd.collect())
```

Relationship Between SparkSession and SparkContext

SparkSession



When you create a `SparkSession`, Spark automatically creates (or reuses) a `SparkContext`.

```
spark = SparkSession.builder.getOrCreate()

sc = spark.sparkContext
```

So, every `SparkSession` contains a `SparkContext`, but `SparkContext` does not contain a `SparkSession`.

Responsibilities

SparkSession

- Entry point for all Spark applications.
 - Creates `DataFrames`.
 - Executes SQL queries.
 - Reads and writes data.
 - Manages Spark configurations.
 - Provides access to `SparkContext`.
 - Supports Structured Streaming.
 - Works with `MLlib`.
-

SparkContext

- Connects the application to the Spark cluster.
- Creates `RDDs`.
- Sends jobs to the cluster manager.
- Schedules tasks.
- Communicates with executors.
- Monitors job execution.
- Collects results back to the driver.

Key Differences

Feature	SparkSession	SparkContext
Introduced	Spark 2.0	Spark 1.x
Purpose	Unified entry point for all Spark APIs	Core execution engine
Main API	DataFrame, SQL, Streaming, MLlib	RDD
Creates DataFrames	✓ Yes	✗ No
Executes SQL	✓ Yes	✗ No
Creates RDDs	Indirectly (through <code>spark.sparkContext</code>)	✓ Yes
Connects to Cluster	Through SparkContext	✓ Yes
Used in Modern Spark	✓ Yes	Used mainly for RDDs
Contains the Other?	Contains/accesses SparkContext	Does not contain SparkSession

Example

Using SparkSession (Recommended)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Demo").getOrCreate()

df = spark.read.csv("employees.csv", header=True)

df.show()
```

Using SparkContext (RDD)

```
sc = spark.sparkContext

numbers = sc.parallelize([1, 2, 3, 4, 5])

squared = numbers.map(lambda x: x * x)

print(squared.collect())
```

Which One Should You Use?

- Use **SparkSession** for almost all modern Spark applications.
- Use **SparkContext** only when you need low-level RDD operations or legacy RDD-based code.

In practice, most PySpark applications start with a **SparkSession**, and **SparkContext** is accessed only when RDD functionality is required.

Interview Questions

Q1. Why was SparkSession introduced?

Before Spark 2.0, developers had to create multiple contexts:

- **SparkContext**
- **SQLContext**
- **HiveContext**

SparkSession unified these into a single entry point, making Spark applications simpler and easier to develop.

Q2. Does SparkSession replace SparkContext?

No.

SparkSession does not replace SparkContext. It wraps and manages it.

When you create a **SparkSession**, Spark automatically creates or reuses a **SparkContext** internally.

Q3. Can you create an RDD using SparkSession?

Not directly.

You access the underlying **SparkContext**:

```
rdd = spark.sparkContext.parallelize([1, 2, 3])
```

Q4. Can SparkContext execute SQL queries?

No.

SparkContext only supports **RDD operations**. SQL queries require SparkSession.

Q5. Which one is the entry point of Spark?

- **Spark 1.x:** SparkContext
 - **Spark 2.x and later:** SparkSession
-

Interview Summary

- **SparkSession** is the modern, unified entry point to Apache Spark.
- It supports **DataFrames, Spark SQL, Structured Streaming, MLlib, and provides access to SparkContext**.
- **SparkContext** is the core execution engine that connects the application to the cluster and is primarily responsible for **RDD operations, job scheduling, and task execution**.
- In modern Spark applications, create a **SparkSession** first, and use **SparkContext** only when RDD-specific functionality is needed.